

Boundary Comparators: What Non-Linux Systems Teach

Boundary Comparators: What Non-Linux Systems Teach I

The candidates examined so far live inside the Linux workstation and server lanes. This section places seven systems that sit outside those lanes — privacy distributions, a non-Linux BSD, formally verified and capability-based microkernel systems, a mobile platform, and offensive-toolkit distributions — into the same property vocabulary. Their value here is comparative: each one demonstrates, in a deployed artifact, a property that the composition argument of this review needs, and each also demonstrates the boundary at which its claim stops. Reading them as competitors for a single “most secure system” title misses what they actually teach. All entries follow the evaluation discipline of sec. 4: documented designs, explicitly scoped claims, no numeric scores.

Boundary Comparators: What Non-Linux Systems Teach I

This lane also moved during the review window. OpenBSD shipped two releases and, more tellingly, used its errata process to *remove* a previously shipped mitigation; seL4's verification frontier extended to a new architecture and a new proof class; Sculpt 26.04 reworked how platform drivers obtain authority; and Tails rebuilt on a new Debian base while preserving its persistence model across automatic upgrades. For systems whose value is precisely their engineering discipline, release mechanics are evidence, not trivia (O. Project 2026b; seL4 Foundation 2026c; Labs 2026; T. Project 2026c).

Boundary Comparators: What Non-Linux Systems Teach I

Table 1: Non-Linux and boundary comparators: documented contribution, boundary or caveat, and the lesson each contributes to the composition argument. Assessments are analytical judgments from documented designs and project documentation.

System	Documented contribution	Boundary or caveat	Lesson for the composition argument
Whonix	A gateway/workstation split that routes all traffic through a dedicated Tor gateway; its workstation documentation explicitly warns that compromise exposes the workstation's credentials and browser data (W. Project 2026b, 2026c). Whonix 17 has reached end of security support (with final deprecation notices through early and mid 2026), making Whonix 18 the only supported line (W. Project 2026a).	Anonymity is not credential protection: the project's own documentation treats workstation compromise as exposing workstation-held secrets. Running a deprecated release line to avoid migration is exactly the maintenance failure the supported-line discipline warns against.	Boundary placement of networking — keeping the network-facing component in a separate trust domain — is a structural pattern that composes with compartmentalization; it does not make an authorized agent harmless.
Tails	An amnesic live environment with Tor networking, plus optional encrypted Persistent Storage whose trade-offs the project documents (T. Project 2026a, 2026b). The 7.x series is built on the Debian 13 base, preserves Persistent Storage across its automatic upgrade path, and current releases (7.12) carry a documented notice about Secure Boot	Amnesia addresses trace reduction and session reset, not information exposed while the session is live; enabling persistence deliberately reintroduces state that must then be protected. Even an amnesic system's boot chain needs certificate maintenance.	Session-state hygiene is one contribution to persistence, <code>e_recovery</code> , and it must not be confused with a recovery plan for credentials used during the session.

Boundary Comparators: What Non-Linux Systems Teach II

OpenBSD

Documented secure-by-default service choices, privilege separation, and exploit mitigations including `W^X` and system-wide hardening (O. Project 2026d, 2026a). Releases 7.8 (2025-10-22) and 7.9 (2026-05-19) document the current state: a new `__pledge_open(2)` interface refined the pledge-to-open path, and `pinsyscall` gained a `PT_LOAD` refusal check that prevents mapping crafted executable segments over pinned entry points; kernel guard pages back the discipline (Project 2025, 2026c).

2026 errata for 7.8 and 7.9 *removed* the previously documented `tmppath` permission and fixed a related namei race — the project retracted one of its own mitigations when its maintenance burden and race behavior outweighed its value (O. Project 2026b). The documented record also does not establish superiority for a modern browser-heavy, GPU-dependent, AI-development workstation.

Disciplined trusted-computing-base design is a property, not a platform identity; its techniques transfer as ideas even where the OS itself does not fit the workload. Even the most disciplined base revises its own mitigations through a public errata process.

seL4

Formal verification of a small microkernel with explicitly scoped proofs and assumptions, including boot, hardware, and DMA-related qualifications (seL4 Foundation 2026a; Klein et al. 2009). The 2026 seL4 milestone set documents the seL4 16.0.0 release, completion of the functional verification of the MCS kernel on 64-bit RISC-V, and the first confidentiality proof on

The proof boundary does not extend to the browser, driver stack, firmware chain, or user workflow that a real deployment adds on top. New proofs on new architectures broaden the frontier; they do not lift the existing scope caveats.

Strong assurance of a small core is one ingredient of the composition; every verification claim must be read at its stated boundary, not at the boundary a reader hopes for.

Boundary Comparators: What Non-Linux Systems Teach III

Genode / Sculpt

Sculpt describes a general-purpose OS built from Genode's microkernel architecture with capability-based security, sandboxed drivers, and VMs, at a 26.04 release and in day-to-day use by its developers. The 26.04 release (2026-04-30) adds a platform-driver gatekeeper that mediates which drivers may claim which devices, IOMMU-based DMA confinement, and a "black hole" component that swallows device-access requests from components without matching capability grants (Labs 2026; Genode Foundation 2026).

seL4 proof claims must not be transferred automatically to every Genode/Sculpt configuration or to the applications it hosts.

The most active capability-based desktop trajectory in this review — the lineage of (Levy 1984) — worth watching as a structural direction rather than treated as a deployable comparator for hostile desktop workloads today.

GrapheneOS

Hardening atop Android's security model, documented to keep even Google Play services inside the standard app sandbox (GrapheneOS Foundation 2026a). The 2026 release documentation adds per-application opt-outs from `hardened_malloc` for compatibility, a five-mode protection model for the USB-C pogo pins against malicious accessories, and continued hardening of the

A mobile comparator, not a desktop Linux replacement; its guarantees are scoped to its platform. The per-app allocator opt-outs are themselves evidence of compatibility pressure: even the most defaults-disciplined mobile platform keeps an escape hatch.

The transferable lesson is shipping useful applications with constrained authority by default — mainstream evidence that application `_confinement` with usable defaults is an achievable product property, not an aspiration.

Boundary Comparators: What Non-Linux Systems Teach IV

Kali / Parrot

Kali identifies penetration testing and security auditing as its purpose (Linux 2026); Parrot documents hardening measures while stating its core remains tuned for security and forensics (Security 2026).	Offensive-tool availability is not evidence of superior protection for the machine running those tools; both are purpose-built workloads.	The label fallacy in its purest form: a distribution's purpose statement constrains what its design should be credited with, exactly as sec. 4 argues for property-based rather than name-based assessment.
---	---	---

What each comparator contributes to the composition argument I

The composition argument of this review holds that the security properties that matter — containment, authority limitation, integrity, recoverability, update operations, supply-chain trust, usable delegation — are supplied by different components cooperating, not by any single system name. Each comparator earns its place by isolating one term of that argument.

What each comparator contributes to the composition argument I

Whonix contributes the networking boundary. Its gateway/workstation structure is the same design move as Qubes keeping networking out of the privileged administrative domain sec. 5, demonstrated in a privacy-focused deployment: the component that meets the hostile network is a separate, disposable-trust domain (W. Project 2026b). Its documented compromise-exposure warning is equally instructive: anonymity infrastructure does not protect the workstation's own credentials and browser data from a compromised workstation (W. Project 2026c). The project's end-of-security-support notice for Whonix 17 — with deprecation notices arriving across early and mid 2026 — adds the maintenance dimension: a boundary design is only as good as its supported line, and the project treats migration to Whonix 18 as a user obligation, not an option (W. Project 2026a). Anonymity and credential protection are distinct properties; confusing

What each comparator contributes to the composition argument I

them produces designs that hide the operator's location while surrendering the operator's secrets.

Tails contributes state hygiene. Amnesia is a deliberate persistence_recovery strategy: the default system leaves nothing to forensically clean and resets every session. The optional Persistent Storage shows the honest tension in that design — every byte an operator chooses to persist is a byte that survives compromise and must be protected by other means (T. Project 2026b). The 7.x series on its Debian 13 base demonstrates that the hygiene can be sustained across upgrades rather than sacrificed to them: automatic upgrades now preserve the Persistent Storage, so the amnesic posture and an update path no longer trade off (T. Project 2026c). The same release line documents a Secure Boot certificate-expiry notice for users of the earlier 7.7-era Secure Boot enablement — a reminder that even an

What each comparator contributes to the composition argument I

amnesic system has a boot chain with its own maintenance clock. For the composition argument, Tails demonstrates that reducing what survives is a legitimate alternative to defending what persists, and that the two strategies must be chosen knowingly rather than mixed accidentally.

OpenBSD contributes trusted-base discipline — including self-revision. Privilege separation, W^X , and secure-by-default service configuration are documented properties of a codebase maintained under an unusual security-first discipline (O. Project 2026d, 2026a). The 2026 record sharpens the lesson. The errata process for releases 7.8 and 7.9 removed the pledge `tmppath` permission across a series of errata and fixed a related `namei` race — retracting a documented mitigation whose race behavior made it a liability rather than an asset (O. Project 2026b). Release 7.9 then introduced `__pledge_open(2)`, which narrows the window between a pledged

What each comparator contributes to the composition argument I

path check and the actual open, and strengthened `pinsyscall` to refuse `PT_LOAD` mappings that would override pinned entry points (O. Project 2026c). A trusted base that removes its own mitigations through a public, dated errata trail demonstrates the property that matters: the mitigation inventory is a maintained engineering artifact, not an advertising list. The correct comparative conclusion remains narrow: for narrowly scoped network services, OpenBSD is a serious non-Linux comparator; for a browser-heavy development workstation with GPU acceleration and AI tooling, the documented evidence does not support a superiority claim. Its lessons — small trusted base, mitigations as defaults, mitigations subject to revision — transfer to the composition argument as design standards for whichever platform actually runs the workload.

What each comparator contributes to the composition argument I

seL4 and Genode/Sculpt contribute the small-trusted-base and capability-mediation trajectories. seL4's formal verification demonstrates that a practical microkernel can carry machine-checked guarantees — and its assumptions documentation shows how verification claims must be scoped to boot, hardware, and DMA assumptions (seL4 Foundation 2026a; Klein et al. 2009). The 2026 record shows the frontier still moving in two directions at once: the functional verification of the MCS kernel completed on 64-bit RISC-V, and the first confidentiality proof landed on AArch64 — a non-interference property on the architecture real phones and laptops run (seL4 Foundation 2026c); Microkit 2.3.0 adds x86_64 IOMMU support to the smaller deployment path, acknowledging that even a small verified kernel must delegate device containment to hardware (seL4 Foundation 2026b). Genode and Sculpt demonstrate the same lineage's userland expression, and the 26.04

What each comparator contributes to the composition argument I

release makes the capability argument concrete at the hardware boundary: the platform-driver gatekeeper mediates which components may claim which devices, the IOMMU confines what a driver's DMA can actually reach, and the “black hole” component absorbs device-access requests from components holding no matching capability — a structural refusal of the confused-deputy path rather than a policy warning about it (Labs 2026; Levy 1984). Capability mediation matters directly to agents: a capability system is the classic answer to the confused-deputy problem of an authority holder that cannot distinguish legitimate from attacker-chosen requests (Hardy 1988). The caveat cuts both ways: proof claims attach to configurations, not product names, and a capability system hosting a compromised browser gains no automatic protection for what the browser legitimately receives.

What each comparator contributes to the composition argument I

GrapheneOS contributes the shipped-defaults demonstration — and its limits. A mainstream mobile platform that keeps even first-party services inside the standard app sandbox shows that constrained authority by default is a product decision, not a research artifact (GrapheneOS Foundation 2026a). The 2026 documentation adds three details that are as instructive as the headline claim: per-application opt-outs from `hardened_malloc` exist because some applications fail under the hardened allocator, a five-mode protection model now governs the USB-C pogo pins against malicious accessories, and the sandboxed-Play environment continues to be hardened in place (GrapheneOS Foundation 2026b). Each item is evidence of the same design stance — containment defaults with narrowly scoped, documented escape hatches — and of its cost: even the most defaults-disciplined platform in this review maintains compatibility

What each comparator contributes to the composition argument I

exceptions, which is exactly the maintenance burden the desktop candidates of sec. 7 must budget for. The mobile platform is not the answer for a workstation; it is evidence that the property is shippable.

Kali and Parrot contribute the negative control. Their documented purposes are offensive tooling and forensics (Linux 2026; Security 2026), and neither project claims otherwise. Including them makes the label fallacy vivid: installing an offensive toolkit does not harden the machine that runs it, any more than a firewall vendor's marketing hardens a desktop. Assessment must attach to documented properties of the deployed system, not to the security connotations of a name.

What each comparator contributes to the composition argument I

Taken together, the comparators reinforce the review's central claim rather than competing with it. None of them, alone, addresses both failure paths of sec. 3: Whonix and Tails narrow the exploitation exposure of networking and state but leave authorized agent access untouched; seL4 and Genode narrow the trusted base without mediating an agent's credentials; GrapheneOS constrains applications without constraining what a persuaded agent may do with legitimate access. The synthesis — which components, composed, cover both paths — is the trust-domain architecture of sec. 10; the scenario-level placement of these comparators appears in sec. 16.